

Operating Systems and Kernel Design (EECE 4811/EECE 5811)
Fall 2023
Prof. Megherbi

0.1 Connecting to the Anaconda cluster

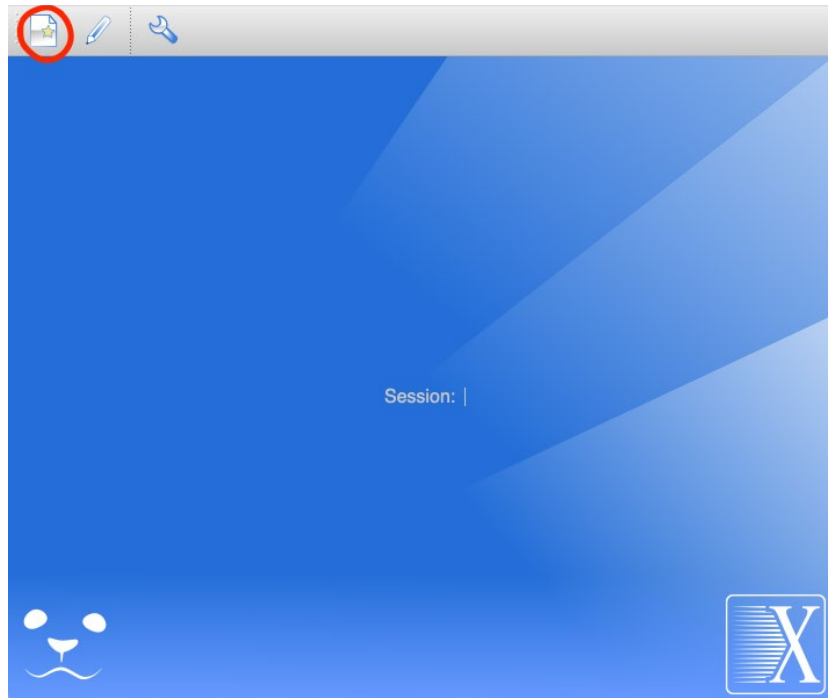
This course provides over two dozen Linux hosts for you to edit and execute your code on. To connect to it, you must either (1) be on campus or (2) connect to campus via UMass Lowell's VPN. More information on UML's VPN can be found here: <https://www.uml.edu/IT/Services/Get-connected/Remote-Access/default.aspx>

Your cluster login credentials are the **same** as your UML email, SiS, and BlackBoard credentials. The following examples for connecting to the cluster will use "Firstname_Lastname@student.uml.edu" and "pass123" for credentials; substitute these with your credentials when you log on.

You have three methods for connecting to the cluster: (1) physically be in Ball 410 at a computer, (2) the X2Go client, or (3) SSH. The X2Go client offers a graphical user interface (including text editors), while SSH offers terminal access.

0.1.1 Connecting with X2Go

- (1) Download and install X2Go client: <http://wiki.x2go.org/doku.php/download:start>
- (2) Select "New Session":



- (3) Enter a host (anaconda2.uml.edu ~ anaconda32.uml.edu), your email address, and select “MATE” for the Session type:

Connection | Input/Output | Media | Shared folders

Session name:

 << change icon

Path: ...

Server

Host:

Login:

SSH port:

Use RSA/DSA key for ssh connection: 

☐ Try auto login (via SSH Agent or default SSH key)

☐ Kerberos 5 (GSSAPI) authentication

☐ Delegation of GSSAPI credentials to the server

☐ Use Proxy server for SSH connection

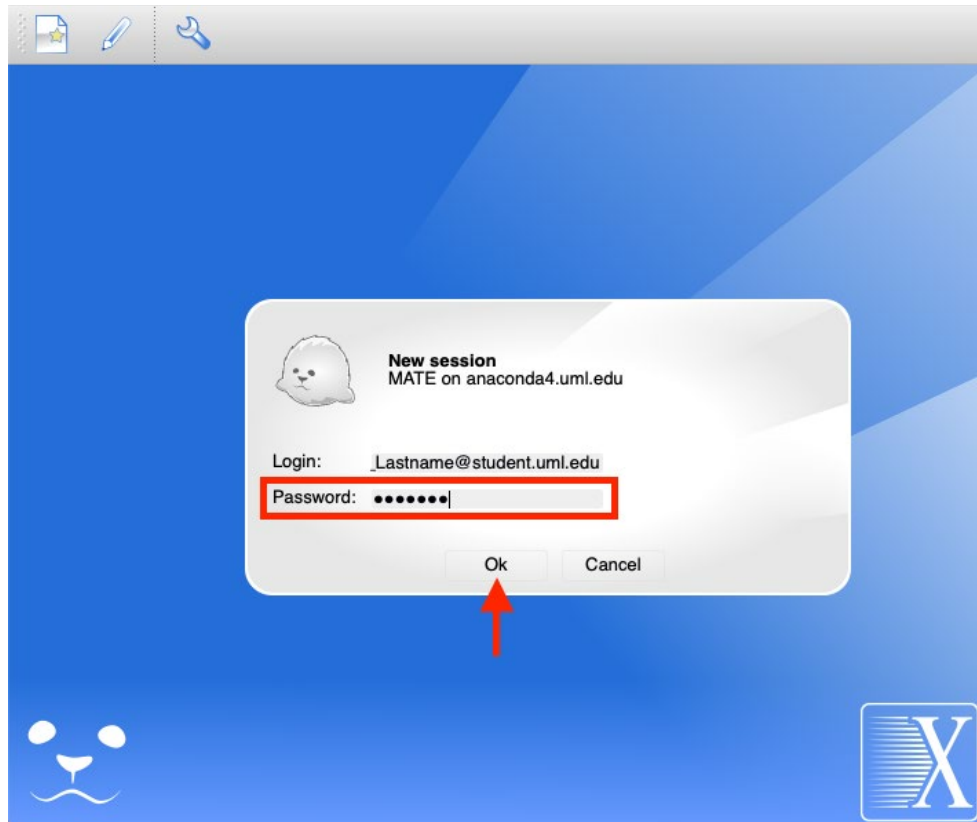
Session type

☐ Run in X2GoKDrive (experimental)

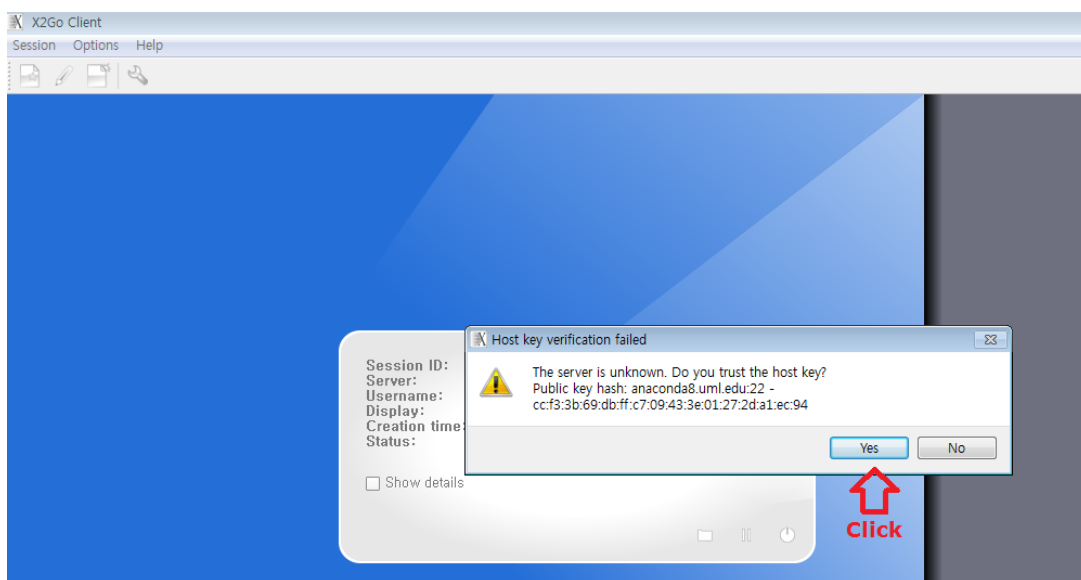
Command:



(4) Click on the new session, enter your UML password, and click “Ok”:



(5) Click yes if you are prompted for whether you should trust the host key:

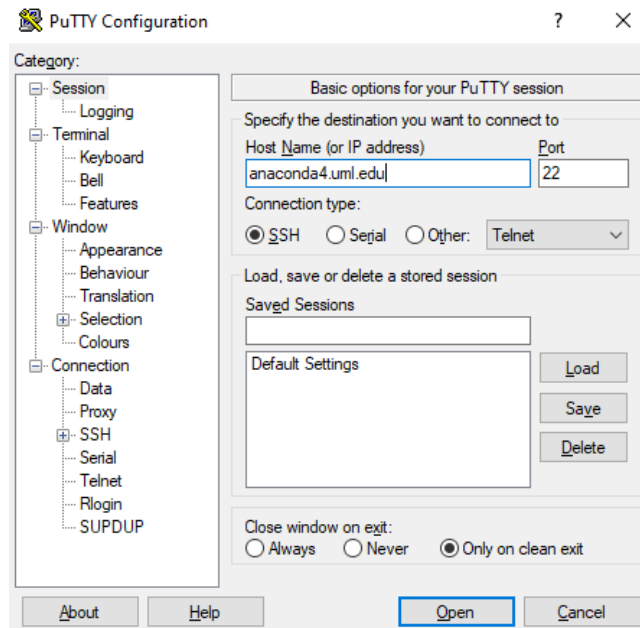


That's it! You are now connected to Ball 410 with a graphical user interface. If you are having trouble connecting, try a different host, or wait a few minutes.

0.1.2 Connecting via SSH

0.1.2.1 On Windows

The recommended SSH client for Windows is “PuTTY”: <https://putty.org/>. Enter the desired host, click “Open”, and enter your UML credentials:



0.1.2.2 On Mac/Linux

All Linux and Mac hosts (should) have SSH already installed. Once you are connected to UML’s VPN, open a local terminal, and:

- (1) Execute “ssh Firstname_Lastname@student.uml.edu@anaconda4.uml.edu”, substituting your credentials and desired host
- (2) Type “yes” to accept the Anaconda host as a known, trusted host
- (3) Enter your UML password

```
~ % ssh Firstname_Lastname@student.uml.edu@anaconda4.uml.edu
The authenticity of host 'anaconda4.uml.edu (10.53.4.12)' can't be established.
ED25519 key fingerprint is SHA256:kLqTeQhmNoGsus3k0nkvsslm0f8D3V47YaChxheKzc.
This host key is known by the following other names/addresses:
~/.ssh/known_hosts:40: anaconda3.uml.edu
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'anaconda4.uml.edu' (ED25519) to the list of known hosts.
Firstname_Lastname@student.uml.edu@anaconda4.uml.edu's password: [REDACTED]
```

Connecting, especially the first time, may take some time (~30 seconds). Just be patient.

0.2 Navigating Linux

If SSH is your desired connection method, you will need to know how to navigate the terminal/console, execute commands, and manipulate files without a graphical user interface.

A cheat sheet for Linux commands can be found here: <https://linuxconfig.org/linux-commands-cheat-sheet>

0.3 Available tools

0.3.1 General purpose text editors

There are two general purpose graphical text editors available on the Anaconda cluster: MATE's default "Text Editor", and "Pluma", a fork of Gedit. Both have the same functionality, e.g., saving and loading files, multiple files in tabs, etc., however Pluma also offers source code highlighting and auto indentation.

More information on Pluma can be found here: <https://wiki.mate-desktop.org/mate-desktop/applications/pluma/>

0.3.2 Vim

The learning curve for Vim is steep, however it has its advantages. One such advantage is being able to be run inside a terminal/console (e.g., over PuTTY), making it appealing to work with on remote machines.

A crash course in Vim is available here: <https://effective-shell.com/part-6-advanced-techniques/a-vim-crash-course/>

0.3.3 Compiling and debugging

Ensure you can access the compiler, GCC, with the following command:

```
~ % gcc --version
```

Usage examples of GCC, including compiling a C file, are available here: <https://www.geeksforgeeks.org/gcc-command-in-linux-with-examples/>

This course will require the use of Makefiles. Information on how Makefiles are constructed and used is available here: <https://makefiletutorial.com/>. An example of a makefile is shown below:

```
all: main.o help.o
    gcc -o all main.o help.o

main.o : main.c help.h
    gcc -c main.c

help.o: help.c help.h
    gcc -c help.c
```

List of all compilation tasks

First file to compile (depends on both “main” and “help”)

Second file to compile (only depends on “help”)

The tool available for debugging is the GDB debugger. More information on the GDB debugger is available here: <https://www.sourceware.org/gdb/documentation/>

See class schedule document for exam dates and lab assignment /project due deadlines (and when they are out).

See the document “Guide_for_Submission_and_Report_structure_fall-2023” for report content guidelines.

BL410_VPN_X2GO_OTHER-Guidelines_F_2023-megherbi-os